Week 2 - Plan

- Introduction to DBs, MongoDB, and Mongoose, Atlas, working with AI, and plan creating an app with AI.
- Threadhive - explanation of reddit, subreddit and threads
- Data is central to any app and the first thing we usually design is the data model.
- What is a database?
- Explanation of **threadive-backend-starter**
  - Open the exact folder in VSCode
  - Explain the folder structure, files and packages used
  - `npm install` to install dependencies - make sure to be in project folder (*cwd*)
  - Run commands in **cmd** and NOT in **powershell**
- Login to MongoDB Atlas, and create a project, and then a cluster
  - Create a database user (**note it down**!), and whitelist your IP address
  - understand the connection string and note it down in the **.env** file
    - add the database name at the end of the URI
  - Understand **.env** file and how to use it
  - **.env** file should be in the project root folder and should not be committed to git
  - Why use env files and not hardcode the connection string in the code?
  - About dotenv package and how to use it
- Run `npm run queries` and check if it connects to the database
- Run `npm run populate` and check if it *seeds* the database
  - Understand the script
  - About Mongoose and understanding schemas and models in the app
    - Why structure is good even for NoSQL databases
    - References to other documents in the database
- Write the queries and run and test
  - Query 1

```
const user = await User.findOne({
    email: 'diana@example.com'
});
```

  - Query 2

```
const programmingSubreddit = await Subreddit.findOne({
    name: 'programming'
});
const threads = await Thread.find({
    subreddit: programmingSubreddit._id
});
```

  - Query 3

```
const ethan = await User.findOne({
    name: 'Ethan'
});
const ethanThreads = await Thread.find({
    author: ethan._id
});

// How to do this one query without doing two separate queries? Use aggregation
pupeline's $lookup operator
const threads = await Thread.aggregate(
    [
        {
            $lookup: {
                from: "users",
                localField: "author",
                foreignField: "_id",
                as: "author"
            }
        },
        { $unwind: "$author" },
        { $match: { "author.name": "Ethan" } }
    ]
);

// Alternatively,
// 1. This returns all threads. For non-Ethan authors, author becomes null.
const threads = await Thread
    .find()
    .populate({
        path: "author",
        match: { name: "Ethan" }
    });

// 2. So filter after populate:
const ethanThreads = threads.filter(thread => thread.author !== null);
```

- Query 4

```
const authorIds = await Thread.distinct("author");
const authors = await User.find({
    _id: { $in: authorIds }
});
```

- Query 5

```
const threads = await Thread.find({
    upvotes: { $gte: 2 }
})
```

- Query 6

```
const afterJan2024Threads = await Thread.find({
    createdAt: { $gte: new Date('2024-01-01') }
});
```

- Query 7

```
const subredditDevops = Subreddit.findOne({
    name: 'devops'
});
const userEthan = await User.findOne({
    name: 'Ethan'
});
const thread = await Thread.create({
    author: userEthan._id,
    subreddit: subredditDevops._id,
    title: 'Test',
    content: 'This is a test thread',

    // These have defaults
    // upvotes: 0,
    // downvotes: 0,
    // voteCount: 0,

    createdAt: new Date()
});
```

- Query 8

```
const thread = await Thread.findOne({
    title: 'Docker and Kubernetes'
});
if ( thread ) {
    thread.title = 'Updated title';
    await thread.save();
} else {
    console.log('Thread not found');
}
```

- Query 9

```
const userEthan = await User.findOne({
    name: 'Ethan'
});
const deleted = await Thread.deleteMany({
    author: userEthan._id
});
console.log(`Deleted ${deleted.deletedCount} threads`);
```

- Query 10

```
const subreddits = await Subreddit.find();

for ( const subreddit of subreddits ) {
    await Thread.deleteMany({
        subreddit: subreddit._id
    });
    await Subreddit.deleteOne({
        _id: subreddit._id
    });
}

// Alternatively, we can delete ALL the threads and ALL the subreddits in one go using deleteMany without any filter. This is a destructive operation, so be careful!
await Thread.deleteMany({});
await Subreddit.deleteMany({});
```

- Query 11

```
const result = await Thread.aggregate([
    {
        $group: {
            _id: "$author",
            count: { $sum: 1 }
        }
    },
]);
```

- Query 12

```
const result = await Thread.aggregate([
    {
        $group: {
            _id: "$author",
            count: { $sum: 1 }
        }
    },
    {
        $sort: { count: -1 }
    },
    {
        $limit: 1
    }
]);
```

- Commit the changes to git and push to GitHub (create new repository if needed - show using GitHub UI instead of VSCode)
- System Prompt in GitHub Copilot
  - What is System prompt in GitHub Copilot
    - Show leaked system prompts of LLMs
  - In Week 9, we will design apps where we provide system prompts

- Rebuilding the models with AI
  - **IMPORTANT**: Start a new session
  - Open fresh folder and run `npm init -y` to create a new project
    - Use `"type": "module"` in `package.json` to use ES modules
  - Install dependencies: `npm install mongoose dotenv`
  - Create a `.env` file to store environment variables
  - Create a `models` folder to define Mongoose schemas
  - Provide the prompt to GitHub Copilot to generate Mongoose models for `User`, `Subreddit`, and `Thread` (provide field, type and constraints)
    - **NOTE**: Shift + Enter for new line in GitHub Copilot prompt
  - Make sure to review the generated code and make necessary adjustments
- Explain Context Window and monitoring it in GitHub Copilot
  - Explain Cached tokens
  - Emphasize using new session for each new prompt to avoid cached tokens
  - Emphasize attaching relevant files/folder in context
- Seed data and script with AI
  - Create seed data for the models using GitHub Copilot
    - **IMPORTANT**: Start a new session
    - Review and adjust the generated code as needed
    - Provide prompt to generate seed data for `User`, `Subreddit`, and `Thread`
      - Provide
        - TASK
        - Context (models, .env)
        - Constraints (number of records, field values, etc.)
        - Output in `data/` folder as `users.json`, `subreddits.json`, and `threads.json`
        - Output seed script in `scripts/seed.js` to clear existing data, read the JSON files and insert data into the database, and log messages for each step
        - Add a script entry in `package.json` to run the seed script: `"seed": "node scripts/seed.js"`
  - Run and test seeding

```
npm run seed
```

- Tests
  - Give the prompts to create tests
    - Create a `tests` folder to write test cases for the models and queries
    - DO NOT use a testing framework - write simple test cases using Node.js and Mongoose
    - Write test cases for each model and query to ensure they work as expected
    - Output in `scripts/tests.js` to run the tests and log messages for each test case
    - Add a script entry in `package.json` to run the tests: `"test": "node scripts/tests.js"`
    - Run the tests and check for any failures or errors
  - Commit the changes to git and push to GitHub
- Using the **Plan mode** for spec-driven development
  - Switch to a reasoning model like Claude Opus 4.6
  - Provide prompt to brainstorm features for the app, and create a plan for the app development
    - MoSCoW prioritization for features
    - simple, medium, complex features with classification and justification
    - features that drive user engagement and retention

        - Answer questions asked
    - Switch to **Agent mode** in and provide prompt to save the MUST and SHOULD features in to features.md file in the project root folder
- **Spec-driven development (decide the plan and then create)**: In a new session in **Agent mode**, ask it to start implementing the features in the plan
    - Prompt: As a senior backend architect and MongoDB expert, help me design an optimal MongoDB schema. I am building a Reddit-like app in MERN stack. I have attached a #file: features.md that contains a detailed list of application features. Your task is to

    i. Carefully analyze the features.

    ii. Identify:
        - The core entities
        - Relationships between entities
        - Expected query patterns (read-heavy vs write-heavy)

    iii. Design an optimal MongoDB schema. For each collection include:
        - Fields
        - Data types
        - Relationships
        - Index recommendations (if any)

    iv. Outline two possible schema designs. Give the pros and cons of each and recommend the best one based on the application features.
    - Review the generated schema design and make necessary adjustments (if needed)
        - Prompt to write out the chosen schema design in Mongoose models in a new folder `db-plan.md`